Hybrid Spectral-Attention TransUNet for Amazon Forest Segmentation

This Colab notebook builds a reproducible **hybrid CNN–Transformer segmentation model** for the Amazon RGB+NIR dataset from **Zenodo record 4498086**. The model combines:

1. a lightweight spectral fusion stem for four Sentinel-2 bands,
2. grouped-plus-pointwise convolutions (HetConv-style) for efficient spatial feature extraction,
3. attention-gated U-Net skip connections, and
4. a transformer bottleneck for long-range context.

The pipeline is organized under Google Drive, downloads and verifies the Zenodo archive, extracts and indexes the raw files, creates an HDF5 cache, trains with augmentation and a BCE+Dice objective, tunes the decision threshold on validation data only, and evaluates once on the held-out test set.

> **Important:** the README supplied with this project reports F1 = 0.9624 for an earlier TransUNet++-style experiment, but this notebook does not copy that number into its results. Run the notebook to obtain a fresh result under the corrected protocol.

Dataset provenance and design choices

Zenodo 4498086 describes Amazon and Atlantic Forest image datasets derived from Sentinel-2 Level 2A imagery. Each image contains bands 4, 3, 2, and 8 stored as byte-valued GeoTIFF data, with associated 512×512 PNG masks. The Amazon split contains 499 training, 100 validation, and 20 test images. The default configuration uses Amazon only; changing one setting enables the Atlantic archive.

The original notes also discuss Zenodo 3233081, a smaller RGB dataset with 30/15/15 train/validation/test images. It remains available as a separate optional configuration, but the central experiment here is the larger four-band Amazon dataset because the additional NIR band is useful for vegetation discrimination.

The raw archive is approximately 1.2 GB. Colab needs additional temporary disk space for the download and extraction. The notebook therefore caches the processed arrays in Drive and skips repeat work when the cache is present.

```
# Cell 1 — install dependencies (run once per Colab runtime)
!pip -q install rasterio h5py scikit-learn seaborn tqdm imageio
!apt-get -qq update && apt-get -qq install -y unrar-free
```

```
W: Skipping acquire of configured file 'main/source/Sources' as repository 'https://r2u.stat.illinois.edu/ubuntu jammy InRel
Selecting previously unselected package unrar-free.
(Reading database ... 122579 files and directories currently installed.)
Preparing to unpack .../unrar-free_1%3a0.0.2-0.1_amd64.deb ...
Unpacking unrar-free (1:0.0.2-0.1) ...
Setting up unrar-free (1:0.0.2-0.1) ...
Processing triggers for man-db (2.10.2-1) ...
```

```
# Cell 2 — imports, reproducibility, and Drive layout
import os, re, json, hashlib, random, shutil, subprocess, sys, time
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image
from tqdm.auto import tqdm
import h5py, rasterio
from sklearn.metrics import confusion_matrix, precision_recall_fscore_support, accuracy_score, jaccard_score
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

SEED = 42
random.seed(SEED); np.random.seed(SEED); tf.random.set_seed(SEED)
try:
    tf.config.experimental.enable_op_determinism()
except Exception:
    pass
print('TensorFlow:', tf.__version__)
print('GPUs:', tf.config.list_physical_devices('GPU'))
```

```
TensorFlow: 2.20.0
GPUs: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

```
# Cell 3 — mount Drive and define a structured project directory
from google.colab import drive
drive.mount('/content/drive')

PROJECT_ROOT = Path('/content/drive/MyDrive/deforestation_segmentation_zenodo')
RAW_DIR = PROJECT_ROOT / 'raw_archives'
EXTRACT_DIR = PROJECT_ROOT / 'extracted'
CACHE_DIR = PROJECT_ROOT / 'cache_h5'
RUNS_DIR = PROJECT_ROOT / 'runs'
PRED_DIR = PROJECT_ROOT / 'predictions'
for p in [RAW_DIR, EXTRACT_DIR, CACHE_DIR, RUNS_DIR, PRED_DIR]: p.mkdir(parents=True, exist_ok=True)
print(PROJECT_ROOT)
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=Tr
/content/drive/MyDrive/deforestation_segmentation_zenodo
```

```
# Cell 4 — experiment configuration
DATASET = 'amazon4band'     # 'amazon4band' or 'atlantic4band'
TARGET_SIZE = (256, 256)    # set to (128,128) for a quick smoke run; (512,512) for full-resolution training
BATCH_SIZE = 4
EPOCHS = 40
LEARNING_RATE = 3e-4
WEIGHT_DECAY = 1e-4
DOWNLOAD_DATA = True
FORCE_REBUILD_CACHE = False

DATASETS = {
    'amazon4band': {
        'archive_name': 'AMAZON.rar',
        'url': 'https://zenodo.org/records/4498086/files/AMAZON.rar?download=1',
        'md5': 'fd2a55b8437d65ac86e1a11161aeef98',
        'extract_name': 'amazon',
    },
    'atlantic4band': {
        'archive_name': 'ATLANTIC FOREST.rar',
        'url': 'https://zenodo.org/records/4498086/files/ATLANTIC%20FOREST.rar?download=1',
        'md5': '86ac319b7ff6738a96f555abb79fb445',
        'extract_name': 'atlantic',
    },
}
CFG = DATASETS[DATASET]
ARCHIVE_PATH = RAW_DIR / CFG['archive_name']
DATA_ROOT = EXTRACT_DIR / CFG['extract_name']
CACHE_PATH = CACHE_DIR / f'{DATASET}_{TARGET_SIZE[0]}px.h5'
print(CFG, '\nCache:', CACHE_PATH)
```

```
{'archive_name': 'AMAZON.rar', 'url': 'https://zenodo.org/records/4498086/files/AMAZON.rar?download=1', 'md5': 'fd2a55b8437d
Cache: /content/drive/MyDrive/deforestation_segmentation_zenodo/cache_h5/amazon4band_256px.h5
```

Download and verify the Zenodo archive

The file URL, MD5 checksum, and dataset counts are taken from the official Zenodo record. A checksum failure stops the notebook instead of silently preprocessing a corrupted archive. The download is stored in Drive so later sessions can reuse it.

```
# Cell 5 — resumable download and MD5 verification
import requests

def md5sum(path, chunk=1024*1024):
    h = hashlib.md5()
    with open(path, 'rb') as f:
        for b in iter(lambda: f.read(chunk), b''): h.update(b)
    return h.hexdigest()

def download_file(url, dest):
    if dest.exists() and dest.stat().st_size > 0:
        print('Archive already exists:', dest, f'({dest.stat().st_size/1e9:.2f} GB)')
        return
    tmp = dest.with_suffix(dest.suffix + '.part')
    with requests.get(url, stream=True, timeout=120) as r:
        r.raise_for_status()
        total = int(r.headers.get('content-length', 0))
        with open(tmp, 'wb') as f, tqdm(total=total, unit='B', unit_scale=True, desc=dest.name) as bar:
            for chunk in r.iter_content(chunk_size=1024*1024):
                if chunk:
                    f.write(chunk); bar.update(len(chunk))
    tmp.rename(dest)

if DOWNLOAD_DATA:
```

```
    download_file(CFG['url'], ARCHIVE_PATH)
    actual = md5sum(ARCHIVE_PATH)
    print('MD5:', actual)
    if actual != CFG['md5']:
        raise RuntimeError(f'MD5 mismatch: expected {CFG["md5"]}, got {actual}')
else:
    print('DOWNLOAD_DATA=False; expecting an existing archive or extracted folder.')
```

```
AMAZON.rar: 100%                                  1.17G/1.17G [12:24<00:00, 1.27MB/s]
MD5: fd2a55b8437d65ac86e1a11161aeef98
```

```
# Cell 6 — extract archive once
if not DATA_ROOT.exists() or not any(DATA_ROOT.rglob('*')):
    DATA_ROOT.mkdir(parents=True, exist_ok=True)
    result = subprocess.run(['unrar', 'x', '-o+', str(ARCHIVE_PATH), str(DATA_ROOT) + '/'], text=True, capture_output=True)
    print(result.stdout[-2000:])
    if result.returncode != 0: raise RuntimeError(result.stderr)
else:
    print('Extraction already exists:', DATA_ROOT)
```

```
tation_zenodo/extracted/amazon/AMAZON/Validation/masks/S2B_MSIL2A_20200202T141649_N0213_R010_T20LQN_20200202T164215_02_07.ti
Extracting  /content/drive/MyDrive/deforestation_segmentation_zenodo/extracted/amazon/AMAZON/Validation/masks/S2B_MSIL2A_202
Extracting  /content/drive/MyDrive/deforestation_segmentation_zenodo/extracted/amazon/AMAZON/Validation/masks/S2B_MSIL2A_202
Extracting  /content/drive/MyDrive/deforestation_segmentation_zenodo/extracted/amazon/AMAZON/Validation/masks/S2B_MSIL2A_202
Extracting  /content/drive/MyDrive/deforestation_segmentation_zenodo/extracted/amazon/AMAZON/Validation/masks/S2B_MSIL2A_202
Extracting  /content/drive/MyDrive/deforestation_segmentation_zenodo/extracted/amazon/AMAZON/Validation/masks/S2B_MSIL2A_202
Extracting  /content/drive/MyDrive/deforestation_segmentation_zenodo/extracted/amazon/AMAZON/Validation/masks/S2B_MSIL2A_202
Extracting  /content/drive/MyDrive/deforestation_segmentation_zenodo/extracted/amazon/AMAZON/Validation/masks/S2B_MSIL2A_202
Extracting  /content/drive/MyDrive/deforestation_segmentation_zenodo/extracted/amazon/AMAZON/Validation/masks/S2B_MSIL2A_202
Extracting  /content/drive/MyDrive/deforestation_segmentation_zenodo/extracted/amazon/AMAZON/Validation/masks/S2B_MSIL2A_202
All OK
```

```
# Cell 7 — discover image/mask pairs using the actual Zenodo 4498086 layout
# Expected layout:
# AMAZON/Training/image + AMAZON/Training/label
# AMAZON/Validation/images + AMAZON/Validation/masks
# AMAZON/Test/image + AMAZON/Test/mask

IMAGE_EXTS = {'.tif', '.tiff'}
MASK_EXTS = {'.png', '.tif', '.tiff'}
IMAGE_DIR_NAMES = {'image', 'images', 'img', 'imgs'}
MASK_DIR_NAMES = {'mask', 'masks', 'label', 'labels', 'groundtruth', 'ground_truth'}


def infer_split_from_path(path):
    split_names = {
        'train': 'train', 'training': 'train',
        'val': 'val', 'valid': 'val', 'validation': 'val',
        'test': 'test', 'testing': 'test',
    }
    for part in Path(path).parts:
        token = part.lower().strip()
        if token in split_names:
            return split_names[token]
    return None


def normalized_filename(path):
    return Path(path).stem.lower().strip()


def discover_pairs(root):
    root = Path(root)
    image_files, mask_files = [], []

    for p in root.rglob('*'):
        if not p.is_file():
            continue
        suffix = p.suffix.lower()
        parent = p.parent.name.lower()
        if parent in IMAGE_DIR_NAMES and suffix in IMAGE_EXTS:
            image_files.append(p)
        elif parent in MASK_DIR_NAMES and suffix in MASK_EXTS:
            mask_files.append(p)

    print('Image files found:', len(image_files))
    print('Mask files found:', len(mask_files))
    if not image_files or not mask_files:
        print('Example files under DATA_ROOT:')
        for p in list(root.rglob('*'))[:40]:
```

```
            print(p)
        raise RuntimeError('No image or mask files found. Check DATA_ROOT.')

    mask_lookup = {}
    for mask_path in mask_files:
        key = (infer_split_from_path(mask_path), normalized_filename(mask_path))
        mask_lookup[key] = mask_path

    pairs = []
    missing = []
    for image_path in sorted(image_files):
        key = (infer_split_from_path(image_path), normalized_filename(image_path))
        mask_path = mask_lookup.get(key)
        if mask_path is None:
            missing.append(image_path)
        else:
            pairs.append((image_path, mask_path))

    if missing:
        print('Images without matching masks:', len(missing))
        print(*missing[:5], sep='\n')
    if not pairs:
        raise RuntimeError('No image-mask pairs found. Inspect the printed paths.')
    return pairs

pairs = discover_pairs(DATA_ROOT)
print('Pairs found:', len(pairs))
print(*pairs[:5], sep='\n')
```

```
Image files found: 619
Mask files found: 619
Pairs found: 619
(PosixPath('/content/drive/MyDrive/deforestation_segmentation_zenodo/extracted/amazon/AMAZON/Test/image/S2A_MSIL2A_20200111T
(PosixPath('/content/drive/MyDrive/deforestation_segmentation_zenodo/extracted/amazon/AMAZON/Test/image/S2A_MSIL2A_20200111T
(PosixPath('/content/drive/MyDrive/deforestation_segmentation_zenodo/extracted/amazon/AMAZON/Test/image/S2A_MSIL2A_20200111T
(PosixPath('/content/drive/MyDrive/deforestation_segmentation_zenodo/extracted/amazon/AMAZON/Test/image/S2A_MSIL2A_20200111T
(PosixPath('/content/drive/MyDrive/deforestation_segmentation_zenodo/extracted/amazon/AMAZON/Test/image/S2A_MSIL2A_20200217T
```

Split detection

The archive is expected to contain folder names such as `train`, `validation`/`val`, and `test`. The helper below uses those names and fails loudly if a split cannot be inferred. It does not randomly reshuffle the official split, which preserves the dataset's intended evaluation protocol.

```
# Cell 8 — infer official split names and save an auditable index

def infer_split_from_path_for_index(path):
    return infer_split_from_path(path)

records = []
for image_path, mask_path in pairs:
    split = infer_split_from_path_for_index(image_path) or infer_split_from_path_for_index(mask_path)
    if split is None:
        raise RuntimeError(f'Could not infer split for {image_path}')
    records.append({'image': str(image_path), 'mask': str(mask_path), 'split': split})

index_df = pd.DataFrame(records)
counts = index_df['split'].value_counts().to_dict()
print('Discovered split counts:', counts)

expected_splits = {'train', 'val', 'test'}
if set(counts) != expected_splits:
    raise RuntimeError(f'Expected splits {expected_splits}, found {set(counts)}')

index_df = index_df.sort_values(['split', 'image']).reset_index(drop=True)
index_df.to_csv(PROJECT_ROOT / f'{DATASET}_file_index.csv', index=False)
print(index_df.groupby('split').size())
```

```
Discovered split counts: {'train': 499, 'val': 100, 'test': 20}
split
test      20
train    499
val      100
dtype: int64
```

```
# Cell 9 — robust GeoTIFF/PNG reading and complete HDF5 cache creation

def read_image(path, target_size=TARGET_SIZE):
    path = Path(path)
    with rasterio.open(path) as src:
```

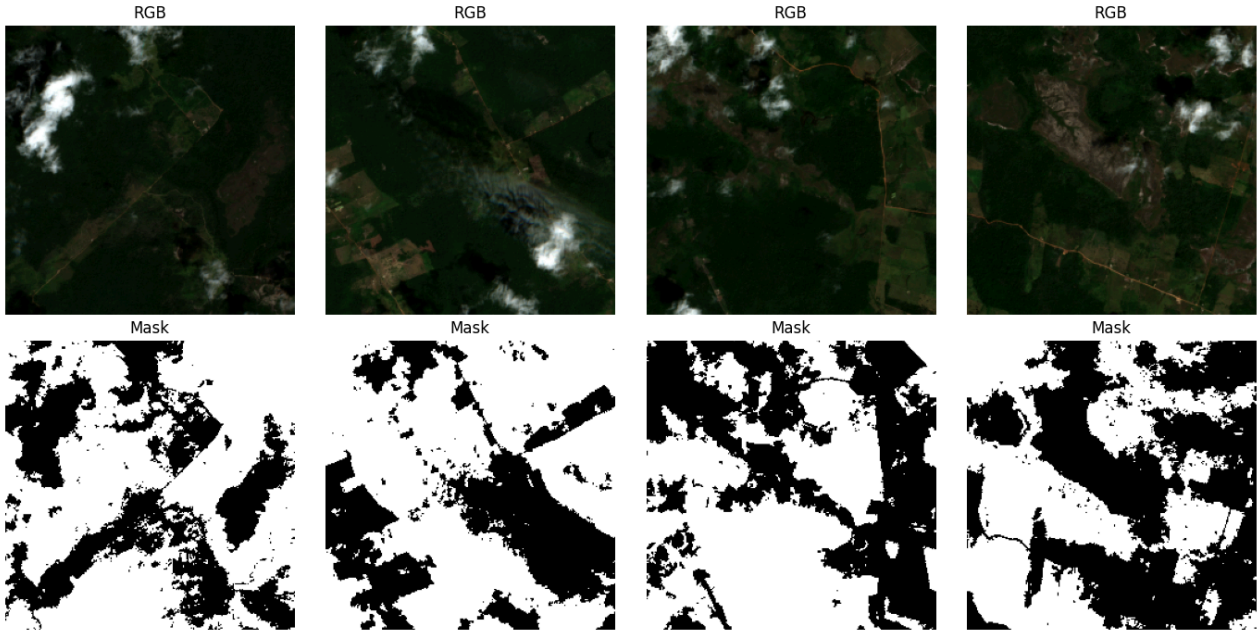
```
        x = src.read()
    x = np.moveaxis(x, 0, -1).astype(np.float32)
    if x.shape[-1] < 4:
        raise ValueError(f'Expected 4 bands, got {x.shape} for {path}')
    x = x[..., :4]
    lo, hi = np.nanpercentile(x, [1, 99])
    x = np.nan_to_num(x, nan=0.0, posinf=hi, neginf=lo)
    x = np.clip((x - lo) / (hi - lo + 1e-6), 0.0, 1.0)
    return tf.image.resize(x, target_size, method='bilinear').numpy().astype('float32')


def read_mask(path, target_size=TARGET_SIZE):
    path = Path(path)
    if path.suffix.lower() in {'.tif', '.tiff'}:
        with rasterio.open(path) as src:
            raw = src.read()
        m = raw[0] if raw.ndim == 3 else raw
    elif path.suffix.lower() == '.png':
        m = np.array(Image.open(path).convert('L'))
    else:
        raise ValueError(f'Unsupported mask format: {path}')

    m = np.asarray(m).astype(np.float32)
    if m.size == 0:
        raise ValueError(f'Empty mask: {path}')
    values = np.unique(m)
    if np.all(np.isin(values, [0, 1])):
        binary = m > 0
    elif np.all(np.isin(values, [0, 255])):
        binary = m > 127
    elif np.all(np.isin(values, [1, 2])):
        binary = m == 2
    else:
        binary = m != values.min()
    binary = tf.image.resize(binary.astype('float32')[..., None], target_size, method='nearest').numpy()[..., 0]
    return (binary > 0.5).astype('uint8')


def build_cache(df, cache_path):
    cache_path = Path(cache_path)
    if cache_path.exists():
        cache_path.unlink()
    with h5py.File(cache_path, 'w') as h:
        for split in ['train', 'val', 'test']:
            h.create_group(split)
            sub = df[df['split'] == split].reset_index(drop=True)
            h.create_dataset(f'{split}/images', shape=(len(sub), *TARGET_SIZE, 4), dtype='float32', compression='lzf')
            h.create_dataset(f'{split}/masks', shape=(len(sub), *TARGET_SIZE, 1), dtype='uint8', compression='lzf')
            for i, row in tqdm(sub.iterrows(), total=len(sub), desc=f'Processing {split}'):
                h[f'{split}/images'][i] = read_image(row['image'])
                h[f'{split}/masks'][i, ..., 0] = read_mask(row['mask'])
        h.attrs['dataset'] = DATASET
        h.attrs['target_size'] = str(TARGET_SIZE)
        h.attrs['source'] = 'https://doi.org/10.5281/zenodo.4498086'


build_cache(index_df, CACHE_PATH)
required = [f'{s}/{k}' for s in ['train', 'val', 'test'] for k in ['images', 'masks']]
with h5py.File(CACHE_PATH, 'r') as h:
    missing = [k for k in required if k not in h]
    if missing:
        raise RuntimeError(f'Cache incomplete. Missing: {missing}')
    for split in ['train', 'val', 'test']:
        print(split, h[f'{split}/images'].shape, h[f'{split}/masks'].shape, 'positive fraction:', float(h[f'{split}/masks'][
print('HDF5 cache created and validated:', CACHE_PATH)
```

```
Processing train: 100%  499/499 [07:03<00:00,  1.11it/s]
Processing val: 100%  100/100 [00:45<00:00,  2.28it/s]
Processing test: 100%  20/20 [00:03<00:00,  6.48it/s]
train (499, 256, 256, 4) (499, 256, 256, 1) positive fraction: 0.52866728941281
val (100, 256, 256, 4) (100, 256, 256, 1) positive fraction: 0.521953125
test (20, 256, 256, 4) (20, 256, 256, 1) positive fraction: 0.5020523071289062
HDF5 cache created and validated: /content/drive/MyDrive/deforestation_segmentation_zenodo/cache_h5/amazon4band_256px.h5
```

```
# Cell 10 — inspect shapes, class balance, and sample tiles
with h5py.File(CACHE_PATH, 'r') as h:
    for split in ['train', 'val', 'test']:
        masks = h[f'{split}/masks']
        print(split, h[f'{split}/images'].shape, masks.shape, 'positive fraction:', float(masks[:].mean()))
    sample_x = h['train/images'][:4]
```

```
    sample_y = h['train/masks'][:4]

fig, ax = plt.subplots(2, 4, figsize=(14, 7))
for i in range(4):
    ax[0, i].imshow(sample_x[i][..., :3]); ax[0, i].set_title('RGB'); ax[0, i].axis('off')
    ax[1, i].imshow(sample_y[i, ..., 0], cmap='gray', vmin=0, vmax=1); ax[1, i].set_title('Mask'); ax[1, i].axis('off')
plt.tight_layout(); plt.show()
```

```
train (499, 256, 256, 4) (499, 256, 256, 1) positive fraction: 0.52866728941281
val (100, 256, 256, 4) (100, 256, 256, 1) positive fraction: 0.521953125
test (20, 256, 256, 4) (20, 256, 256, 1) positive fraction: 0.5020523071289062
```



Model: Spectral-Attention TransUNet

The encoder uses a spectral stem to learn a data-driven RGB/NIR fusion instead of treating the four channels as interchangeable. Each HetConv block adds grouped spatial convolution to a pointwise projection. The bottleneck flattens the deepest feature map into tokens and applies pre-normalized multi-head self-attention. Decoder skip features are filtered by attention gates before concatenation.

The implementation keeps the transformer at the 16×16 bottleneck for 256×256 inputs. This gives global context without the quadratic cost of applying attention to the full image. For 128×128 inputs the bottleneck is 8×8; for 512×512 it is 32×32 and may require a smaller batch size.

```
# Cell 11 — corrected HDF5 input pipeline
# Fixes: h5py cannot directly index with unsorted shuffled indices.

class H5Sequence(keras.utils.Sequence):

    def __init__(
        self,
        path,
        split,
        batch_size,
        augment=False,
        shuffle=False,
        **kwargs
    ):
        super().__init__(**kwargs)

        self.path = str(path)
        self.split = split
        self.batch_size = batch_size
        self.augment = augment
        self.shuffle = shuffle
```

```
        with h5py.File(self.path, "r") as h:
            image_key = f"{split}/images"
            mask_key = f"{split}/masks"

            if image_key not in h:
                raise RuntimeError(f"Missing dataset: {image_key}")

            if mask_key not in h:
                raise RuntimeError(f"Missing dataset: {mask_key}")

            self.n = h[image_key].shape[0]

        self.order = np.arange(self.n)
        self.on_epoch_end()

    def __len__(self):
        return int(np.ceil(self.n / self.batch_size))

    def on_epoch_end(self):
        if self.shuffle:
            np.random.shuffle(self.order)

    def __getitem__(self, idx):

        start = idx * self.batch_size
        end = min((idx + 1) * self.batch_size, self.n)

        ids = self.order[start:end]

        # h5py requires sorted indices for fancy indexing.
        sorted_positions = np.argsort(ids)
        sorted_ids = ids[sorted_positions]

        # Read using sorted indices.
        with h5py.File(self.path, "r") as h:
            x_sorted = h[f"{self.split}/images"][sorted_ids]
            y_sorted = h[f"{self.split}/masks"][sorted_ids].astype("float32")

        # Restore the original shuffled order.
        restore_positions = np.argsort(sorted_positions)

        x = x_sorted[restore_positions]
        y = y_sorted[restore_positions]

        # Apply identical geometric transformations to image and mask.
        if self.augment:

            augmented_images = []
            augmented_masks = []

            for image, mask in zip(x, y):

                combined = tf.concat(
                    [
                        tf.convert_to_tensor(image, dtype=tf.float32),
                        tf.convert_to_tensor(mask, dtype=tf.float32)
                    ],
                    axis=-1
                )

                combined = tf.image.random_flip_left_right(combined)
                combined = tf.image.random_flip_up_down(combined)

                rotation_k = tf.random.uniform(
                    shape=[],
                    minval=0,
                    maxval=4,
                    dtype=tf.int32
                )

                combined = tf.image.rot90(
                    combined,
                    k=rotation_k
                )

                augmented_images.append(combined[..., :4])
                augmented_masks.append(combined[..., 4:])

            x = tf.stack(augmented_images).numpy()
            y = tf.stack(augmented_masks).numpy()
```

```
        return (
            x.astype("float32"),
            y.astype("float32")
        )


# Recreate the generators after redefining the class.
train_seq = H5Sequence(
    CACHE_PATH,
    "train",
    BATCH_SIZE,
    augment=True,
    shuffle=True
)

val_seq = H5Sequence(
    CACHE_PATH,
    "val",
    BATCH_SIZE,
    augment=False,
    shuffle=False
)

test_seq = H5Sequence(
    CACHE_PATH,
    "test",
    BATCH_SIZE,
    augment=False,
    shuffle=False
)

print("Train batches:", len(train_seq))
print("Validation batches:", len(val_seq))
print("Test batches:", len(test_seq))

# Test one batch before starting training.
test_x, test_y = train_seq[0]

print("Test batch image shape:", test_x.shape)
print("Test batch mask shape:", test_y.shape)
print("Image dtype:", test_x.dtype)
print("Mask dtype:", test_y.dtype)
print("Mask values:", np.unique(test_y))
```

```
Train batches: 125
Validation batches: 25
Test batches: 5
Test batch image shape: (4, 256, 256, 4)
Test batch mask shape: (4, 256, 256, 1)
Image dtype: float32
Mask dtype: float32
Mask values: [0. 1.]
```

```
# Cell 12 — model blocks and hybrid model

def hetconv(x, filters, name):
    groups = 4 if filters % 4 == 0 else 1
    a = layers.Conv2D(filters, 3, padding='same', groups=groups, use_bias=False, name=name+'_group')(x)
    b = layers.Conv2D(filters, 1, padding='same', use_bias=False, name=name+'_point')(x)
    x = layers.Add(name=name+'_add')([a,b])
    x = layers.BatchNormalization(name=name+'_bn')(x)
    return layers.Activation('swish', name=name+'_act')(x)

def conv_stack(x, filters, name):
    x = hetconv(x, filters, name+'_1'); x = hetconv(x, filters, name+'_2')
    return x

def attention_gate(skip, gate, filters, name):
    s = layers.Conv2D(filters, 1, padding='same', name=name+'_skip')(skip)
    g = layers.Conv2D(filters, 1, padding='same', name=name+'_gate')(gate)
    q = layers.Activation('relu')(layers.Add()([s,g]))
    a = layers.Conv2D(1, 1, padding='same', activation='sigmoid', name=name+'_coef')(q)
    return layers.Multiply(name=name+'_multiply')([skip, a])

def transformer_bottleneck(x, embed_dim=128, heads=4, mlp_ratio=2, name='transformer'):
    shape = x.shape
    h, w, c = int(shape[1]), int(shape[2]), int(shape[3])
    t = layers.Reshape((h*w, c), name=name+'_tokens')(x)
    t = layers.Dense(embed_dim, name=name+'_projection')(t)
    z = layers.LayerNormalization(name=name+'_ln1')(t)
    z = layers.MultiHeadAttention(num_heads=heads, key_dim=embed_dim//heads, dropout=0.1, name=name+'_mha')(z,z)
    t = layers.Add()([t,z])
```

```
    z = layers.LayerNormalization(name=name+'_ln2')(t)
    z = layers.Dense(embed_dim*mlp_ratio, activation='gelu', name=name+'_mlp1')(z)
    z = layers.Dropout(0.1)(z)
    z = layers.Dense(embed_dim, name=name+'_mlp2')(z)
    t = layers.Add()([t,z])
    t = layers.Reshape((h,w,embed_dim), name=name+'_map')(t)
    return t

def build_model(input_shape=(*TARGET_SIZE,4)):
    inp = keras.Input(input_shape)
    # Spectral fusion: RGB projection + NIR projection + learned gate.
    rgb = layers.Conv2D(16, 1, padding='same', name='rgb_projection')(inp[...,:3])
    nir = layers.Conv2D(16, 1, padding='same', name='nir_projection')(inp[...,3:])
    gate = layers.Activation('sigmoid')(layers.Conv2D(16, 1, padding='same', name='nir_gate')(inp))
    x = layers.Add()([rgb, layers.Multiply()([nir, gate])])
    x = layers.Activation('swish')(x)
    e1 = conv_stack(x, 16, 'enc1')
    e2 = conv_stack(layers.MaxPooling2D(2)(e1), 32, 'enc2')
    e3 = conv_stack(layers.MaxPooling2D(2)(e2), 64, 'enc3')
    e4 = conv_stack(layers.MaxPooling2D(2)(e3), 128, 'enc4')
    b = transformer_bottleneck(layers.MaxPooling2D(2)(e4), 128, 4, 2)
    b = hetconv(b, 128, 'bridge')
    d3 = layers.UpSampling2D(2, interpolation='bilinear')(b)
    d3 = layers.Concatenate()([d3, attention_gate(e4, d3, 64, 'att4')]); d3 = conv_stack(d3, 64, 'dec3')
    d2 = layers.UpSampling2D(2, interpolation='bilinear')(d3)
    d2 = layers.Concatenate()([d2, attention_gate(e3, d2, 32, 'att3')]); d2 = conv_stack(d2, 32, 'dec2')
    d1 = layers.UpSampling2D(2, interpolation='bilinear')(d2)
    d1 = layers.Concatenate()([d1, attention_gate(e2, d1, 16, 'att2')]); d1 = conv_stack(d1, 16, 'dec1')
    d0 = layers.UpSampling2D(2, interpolation='bilinear')(d1)
    d0 = layers.Concatenate()([d0, attention_gate(e1, d0, 8, 'att1')]); d0 = conv_stack(d0, 16, 'dec0')
    out = layers.Conv2D(1, 1, activation=None, name='logits')(d0)
    return keras.Model(inp, out, name='SpectralAttentionTransUNet')

model = build_model()
model.summary()
```

	16)		activation[0][0]
add (Add)	(None, 256, 256, 16)	0	rgb_projection[0… multiply[0][0]
activation_1 (Activation)	(None, 256, 256, 16)	0	add[0][0]
enc1_1_group (Conv2D)	(None, 256, 256, 16)	576	activation_1[0][…
enc1_1_point (Conv2D)	(None, 256, 256, 16)	256	activation_1[0][…
enc1_1_add (Add)	(None, 256, 256, 16)	0	enc1_1_group[0][… enc1_1_point[0][…
enc1_1_bn (BatchNormalizatio…	(None, 256, 256, 16)	64	enc1_1_add[0][0]
enc1_1_act (Activation)	(None, 256, 256, 16)	0	enc1_1_bn[0][0]
enc1_2_group (Conv2D)	(None, 256, 256, 16)	576	enc1_1_act[0][0]
enc1_2_point (Conv2D)	(None, 256, 256, 16)	256	enc1_1_act[0][0]
enc1_2_add (Add)	(None, 256, 256, 16)	0	enc1_2_group[0][… enc1_2_point[0][…
enc1_2_bn (BatchNormalizatio…	(None, 256, 256, 16)	64	enc1_2_add[0][0]
enc1_2_act (Activation)	(None, 256, 256, 16)	0	enc1_2_bn[0][0]
max_pooling2d (MaxPooling2D)	(None, 128, 128, 16)	0	enc1_2_act[0][0]
enc2_1_group (Conv2D)	(None, 128, 128, 32)	1,152	max_pooling2d[0]…
enc2_1_point (Conv2D)	(None, 128, 128, 32)	512	max_pooling2d[0]…
enc2_1_add (Add)	(None, 128, 128, 32)	0	enc2_1_group[0][… enc2_1_point[0][…
enc2_1_bn (BatchNormalizatio…	(None, 128, 128, 32)	128	enc2_1_add[0][0]
enc2_1_act (Activation)	(None, 128, 128, 32)	0	enc2_1_bn[0][0]
enc2_2_group (Conv2D)	(None, 128, 128, 32)	2,304	enc2_1_act[0][0]
enc2_2_point (Conv2D)	(None, 128, 128, 32)	1,024	enc2_1_act[0][0]
enc2_2_add (Add)	(None, 128, 128, 32)	0	enc2_2_group[0][… enc2_2_point[0][…
enc2_2_bn	(None, 128, 128,	128	enc2_2_add[0][0]

(BatchNormalizatio…	32)		
enc2_2_act (Activation)	(None, 128, 128, 32)	0	enc2_2_bn[0][0]
max_pooling2d_1 (MaxPooling2D)	(None, 64, 64, 32)	0	enc2_2_act[0][0]
enc3_1_group (Conv2D)	(None, 64, 64, 64)	4,608	max_pooling2d_1[…
enc3_1_point (Conv2D)	(None, 64, 64, 64)	2,048	max_pooling2d_1[…
enc3_1_add (Add)	(None, 64, 64, 64)	0	enc3_1_group[0][… enc3_1_point[0][…
enc3_1_bn (BatchNormalizatio…	(None, 64, 64, 64)	256	enc3_1_add[0][0]
enc3_1_act (Activation)	(None, 64, 64, 64)	0	enc3_1_bn[0][0]
enc3_2_group (Conv2D)	(None, 64, 64, 64)	9,216	enc3_1_act[0][0]
enc3_2_point (Conv2D)	(None, 64, 64, 64)	4,096	enc3_1_act[0][0]
enc3_2_add (Add)	(None, 64, 64, 64)	0	enc3_2_group[0][… enc3_2_point[0][…
enc3_2_bn (BatchNormalizatio…	(None, 64, 64, 64)	256	enc3_2_add[0][0]
enc3_2_act (Activation)	(None, 64, 64, 64)	0	enc3_2_bn[0][0]
max_pooling2d_2 (MaxPooling2D)	(None, 32, 32, 64)	0	enc3_2_act[0][0]
enc4_1_group (Conv2D)	(None, 32, 32, 128)	18,432	max_pooling2d_2[…
enc4_1_point (Conv2D)	(None, 32, 32, 128)	8,192	max_pooling2d_2[…
enc4_1_add (Add)	(None, 32, 32, 128)	0	enc4_1_group[0][… enc4_1_point[0][…
enc4_1_bn (BatchNormalizatio…	(None, 32, 32, 128)	512	enc4_1_add[0][0]
enc4_1_act (Activation)	(None, 32, 32, 128)	0	enc4_1_bn[0][0]
enc4_2_group (Conv2D)	(None, 32, 32, 128)	36,864	enc4_1_act[0][0]
enc4_2_point (Conv2D)	(None, 32, 32, 128)	16,384	enc4_1_act[0][0]
enc4_2_add (Add)	(None, 32, 32, 128)	0	enc4_2_group[0][… enc4_2_point[0][…
enc4_2_bn (BatchNormalizatio…	(None, 32, 32, 128)	512	enc4_2_add[0][0]
enc4_2_act (Activation)	(None, 32, 32, 128)	0	enc4_2_bn[0][0]
max_pooling2d_3 (MaxPooling2D)	(None, 16, 16, 128)	0	enc4_2_act[0][0]
transformer_tokens (Reshape)	(None, 256, 128)	0	max_pooling2d_3[…
transformer_projec… (Dense)	(None, 256, 128)	16,512	transformer_toke…
transformer_ln1 (LayerNormalizatio…	(None, 256, 128)	256	transformer_proj…
transformer_mha (MultiHeadAttentio…	(None, 256, 128)	66,048	transformer_ln1[… transformer_ln1[…
add_1 (Add)	(None, 256, 128)	0	transformer_proj… transformer_mha[…
transformer_ln2 (LayerNormalizatio…	(None, 256, 128)	256	add_1[0][0]

transformer_mlp1 (Dense)	(None, 256, 256)	33,024	transformer_in2[…
dropout_1 (Dropout)	(None, 256, 256)	0	transformer_mlp1…
transformer_mlp2 (Dense)	(None, 256, 128)	32,896	dropout_1[0][0]
add_2 (Add)	(None, 256, 128)	0	add_1[0][0], transformer_mlp2…
transformer_map (Reshape)	(None, 16, 16, 128)	0	add_2[0][0]
bridge_group (Conv2D)	(None, 16, 16, 128)	36,864	transformer_map[…
bridge_point (Conv2D)	(None, 16, 16, 128)	16,384	transformer_map[…
bridge_add (Add)	(None, 16, 16, 128)	0	bridge_group[0][… bridge_point[0][…
bridge_bn (BatchNormalizatio…	(None, 16, 16, 128)	512	bridge_add[0][0]
bridge_act (Activation)	(None, 16, 16, 128)	0	bridge_bn[0][0]
up_sampling2d (UpSampling2D)	(None, 32, 32, 128)	0	bridge_act[0][0]
att4_skip (Conv2D)	(None, 32, 32, 64)	8,256	enc4_2_act[0][0]
att4_gate (Conv2D)	(None, 32, 32, 64)	8,256	up_sampling2d[0]…
add_3 (Add)	(None, 32, 32, 64)	0	att4_skip[0][0], att4_gate[0][0]
activation_2 (Activation)	(None, 32, 32, 64)	0	add_3[0][0]
att4_coef (Conv2D)	(None, 32, 32, 1)	65	activation_2[0][…
att4_multiply (Multiply)	(None, 32, 32, 128)	0	enc4_2_act[0][0], att4_coef[0][0]
concatenate (Concatenate)	(None, 32, 32, 256)	0	up_sampling2d[0]… att4_multiply[0]…
dec3_1_group (Conv2D)	(None, 32, 32, 64)	36,864	concatenate[0][0]
dec3_1_point (Conv2D)	(None, 32, 32, 64)	16,384	concatenate[0][0]
dec3_1_add (Add)	(None, 32, 32, 64)	0	dec3_1_group[0][… dec3_1_point[0][…
dec3_1_bn (BatchNormalizatio…	(None, 32, 32, 64)	256	dec3_1_add[0][0]
dec3_1_act (Activation)	(None, 32, 32, 64)	0	dec3_1_bn[0][0]
dec3_2_group (Conv2D)	(None, 32, 32, 64)	9,216	dec3_1_act[0][0]
dec3_2_point (Conv2D)	(None, 32, 32, 64)	4,096	dec3_1_act[0][0]
dec3_2_add (Add)	(None, 32, 32, 64)	0	dec3_2_group[0][… dec3_2_point[0][…
dec3_2_bn (BatchNormalizatio…	(None, 32, 32, 64)	256	dec3_2_add[0][0]
dec3_2_act (Activation)	(None, 32, 32, 64)	0	dec3_2_bn[0][0]
up_sampling2d_1 (UpSampling2D)	(None, 64, 64, 64)	0	dec3_2_act[0][0]
att3_skip (Conv2D)	(None, 64, 64, 32)	2,080	enc3_2_act[0][0]
att3_gate (Conv2D)	(None, 64, 64, 32)	2,080	up_sampling2d_1[…
add_4 (Add)	(None, 64, 64, 32)	0	att3_skip[0][0], att3_gate[0][0]

activation_3 (Activation)	(None, 64, 64, 32)	0	add_4[0][0]
att3_coef (Conv2D)	(None, 64, 64, 1)	33	activation_3[0][…
att3_multiply (Multiply)	(None, 64, 64, 64)	0	enc3_2_act[0][0], att3_coef[0][0]
concatenate_1 (Concatenate)	(None, 64, 64, 128)	0	up_sampling2d_1[… att3_multiply[0]…
dec2_1_group (Conv2D)	(None, 64, 64, 32)	9,216	concatenate_1[0]…
dec2_1_point (Conv2D)	(None, 64, 64, 32)	4,096	concatenate_1[0]…
dec2_1_add (Add)	(None, 64, 64, 32)	0	dec2_1_group[0][… dec2_1_point[0][…
dec2_1_bn (BatchNormalizatio…	(None, 64, 64, 32)	128	dec2_1_add[0][0]
dec2_1_act (Activation)	(None, 64, 64, 32)	0	dec2_1_bn[0][0]
dec2_2_group (Conv2D)	(None, 64, 64, 32)	2,304	dec2_1_act[0][0]
dec2_2_point (Conv2D)	(None, 64, 64, 32)	1,024	dec2_1_act[0][0]
dec2_2_add (Add)	(None, 64, 64, 32)	0	dec2_2_group[0][… dec2_2_point[0][…
dec2_2_bn (BatchNormalizatio…	(None, 64, 64, 32)	128	dec2_2_add[0][0]
dec2_2_act (Activation)	(None, 64, 64, 32)	0	dec2_2_bn[0][0]
up_sampling2d_2 (UpSampling2D)	(None, 128, 128, 32)	0	dec2_2_act[0][0]
att2_skip (Conv2D)	(None, 128, 128, 16)	528	enc2_2_act[0][0]
att2_gate (Conv2D)	(None, 128, 128, 16)	528	up_sampling2d_2[…
add_5 (Add)	(None, 128, 128, 16)	0	att2_skip[0][0], att2_gate[0][0]
activation_4 (Activation)	(None, 128, 128, 16)	0	add_5[0][0]
att2_coef (Conv2D)	(None, 128, 128, 1)	17	activation_4[0][…
att2_multiply (Multiply)	(None, 128, 128, 32)	0	enc2_2_act[0][0], att2_coef[0][0]
concatenate_2 (Concatenate)	(None, 128, 128, 64)	0	up_sampling2d_2[… att2_multiply[0]…
dec1_1_group (Conv2D)	(None, 128, 128, 16)	2,304	concatenate_2[0]…
dec1_1_point (Conv2D)	(None, 128, 128, 16)	1,024	concatenate_2[0]…
dec1_1_add (Add)	(None, 128, 128, 16)	0	dec1_1_group[0][… dec1_1_point[0][…
dec1_1_bn (BatchNormalizatio…	(None, 128, 128, 16)	64	dec1_1_add[0][0]
dec1_1_act (Activation)	(None, 128, 128, 16)	0	dec1_1_bn[0][0]
dec1_2_group (Conv2D)	(None, 128, 128, 16)	576	dec1_1_act[0][0]
dec1_2_point (Conv2D)	(None, 128, 128, 16)	256	dec1_1_act[0][0]
dec1_2_add (Add)	(None, 128, 128, 16)	0	dec1_2_group[0][… dec1_2_point[0][…
dec1_2_bn (BatchNormalizatio…	(None, 128, 128, 16)	64	dec1_2_add[0][0]

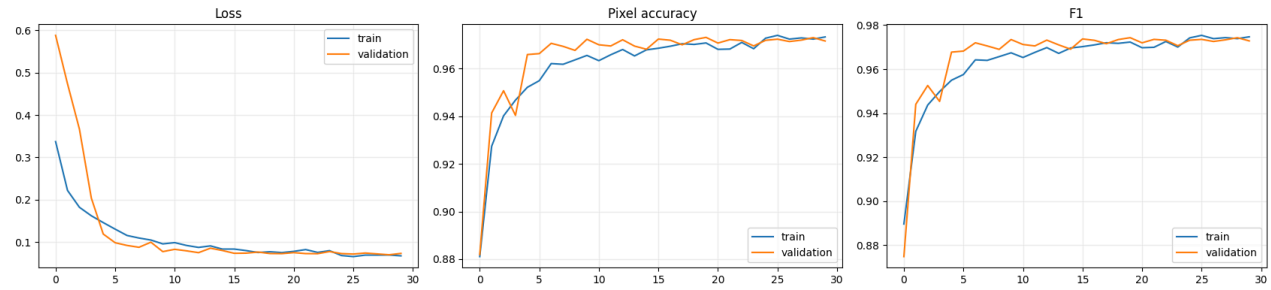
```
# Cell 13 — losses, metrics, optimizer, and callbacks

def dice_loss(y_true, logits, smooth=1.0):
    y_prob = tf.nn.sigmoid(logits)
    axes = (1,2,3)
    inter = tf.reduce_sum(y_true*y_prob, axis=axes)
    denom = tf.reduce_sum(y_true+y_prob, axis=axes)
    return 1 - tf.reduce_mean((2*inter+smooth)/(denom+smooth))

def bce_dice(y_true, logits):
    bce = tf.reduce_mean(tf.nn.sigmoid_cross_entropy_with_logits(labels=y_true, logits=logits))
    return 0.5*bce + 0.5*dice_loss(y_true, logits)


class SegmentationMetrics(keras.metrics.Metric):
    def __init__(self, name='seg_metrics', threshold=0.5, **kwargs):
        super().__init__(name=name, **kwargs); self.threshold=threshold
        self.tp=self.add_weight(name='tp', shape=(), initializer='zeros'); self.fp=self.add_weight(name='fp', shape=(), init
    def update_state(self,y_true,y_pred,sample_weight=None):
        p=tf.cast(tf.nn.sigmoid(y_pred)>=self.threshold,tf.float32); y=tf.cast(y_true>=0.5,tf.float32)
        self.tp.assign_add(tf.reduce_sum(p*y)); self.fp.assign_add(tf.reduce_sum(p*(1-y))); self.fn.assign_add(tf.reduce_sum
    def result(self):
        precision=tf.math.divide_no_nan(self.tp,self.tp+self.fp); recall=tf.math.divide_no_nan(self.tp,self.tp+self.fn)
        return tf.math.divide_no_nan(2*precision*recall,precision+recall)
    def reset_state(self):
        for v in [self.tp,self.fp,self.fn]: v.assign(0.)

try:
    opt = keras.optimizers.AdamW(learning_rate=LEARNING_RATE, weight_decay=WEIGHT_DECAY)
except Exception:
    opt = keras.optimizers.Adam(learning_rate=LEARNING_RATE)
model.compile(optimizer=opt, loss=bce_dice, metrics=[keras.metrics.BinaryAccuracy(name='accuracy', threshold=0.0), Segmentat
RUN_DIR = RUNS_DIR / time.strftime('%Y%m%d_%H%M%S')
RUN_DIR.mkdir(parents=True, exist_ok=True)
callbacks = [
    keras.callbacks.ModelCheckpoint(RUN_DIR/'best.weights.h5', monitor='val_f1', mode='max', save_best_only=True, save_weigh
    keras.callbacks.ReduceLROnPlateau(monitor='val_f1', mode='max', factor=0.5, patience=5, min_lr=1e-6),
    keras.callbacks.EarlyStopping(monitor='val_f1', mode='max', patience=10, restore_best_weights=True),
    keras.callbacks.CSVLogger(RUN_DIR/'history.csv')]
```

```
# Cell 14 — train and save the run configuration

run_config = dict(
    dataset=DATASET,
    target_size=TARGET_SIZE,
    batch_size=BATCH_SIZE,
    epochs=EPOCHS,
    seed=SEED,
    model=model.name
)

(RUN_DIR / "config.json").write_text(
    json.dumps(run_config, indent=2)
)

history = model.fit(
    train_seq,
    validation_data=val_seq,
    epochs=EPOCHS,
    callbacks=callbacks
)

model.save_weights(
    RUN_DIR / "final.weights.h5"
)
```

```
125/125 ━━━━━━━━━━━━━━━━━━━━ 197s 2s/step - accuracy: 0.9274 - f1: 0.9317 - loss: 0.2220 - val_accuracy: 0.9414 - val_f1:
Epoch 3/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 200s 2s/step - accuracy: 0.9402 - f1: 0.9436 - loss: 0.1822 - val_accuracy: 0.9507 - val_f1:
Epoch 4/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 195s 2s/step - accuracy: 0.9468 - f1: 0.9498 - loss: 0.1622 - val_accuracy: 0.9403 - val_f1:
Epoch 5/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 198s 2s/step - accuracy: 0.9521 - f1: 0.9550 - loss: 0.1462 - val_accuracy: 0.9659 - val_f1:
Epoch 6/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 194s 2s/step - accuracy: 0.9549 - f1: 0.9575 - loss: 0.1307 - val_accuracy: 0.9663 - val_f1:
Epoch 7/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 193s 2s/step - accuracy: 0.9621 - f1: 0.9642 - loss: 0.1158 - val_accuracy: 0.9706 - val_f1:
Epoch 8/40
```

```
Epoch 10/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 192s 2s/step - accuracy: 0.9655 - f1: 0.9674 - loss: 0.0959 - val_accuracy: 0.9723 - val_f1:
Epoch 11/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 191s 2s/step - accuracy: 0.9633 - f1: 0.9653 - loss: 0.0989 - val_accuracy: 0.9700 - val_f1:
Epoch 12/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 195s 2s/step - accuracy: 0.9658 - f1: 0.9676 - loss: 0.0922 - val_accuracy: 0.9695 - val_f1:
Epoch 13/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 191s 2s/step - accuracy: 0.9680 - f1: 0.9698 - loss: 0.0876 - val_accuracy: 0.9721 - val_f1:
Epoch 14/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 191s 2s/step - accuracy: 0.9653 - f1: 0.9672 - loss: 0.0912 - val_accuracy: 0.9694 - val_f1:
Epoch 15/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 192s 2s/step - accuracy: 0.9678 - f1: 0.9696 - loss: 0.0837 - val_accuracy: 0.9681 - val_f1:
Epoch 16/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 193s 2s/step - accuracy: 0.9685 - f1: 0.9702 - loss: 0.0837 - val_accuracy: 0.9724 - val_f1:
Epoch 17/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 195s 2s/step - accuracy: 0.9694 - f1: 0.9710 - loss: 0.0801 - val_accuracy: 0.9719 - val_f1:
Epoch 18/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 195s 2s/step - accuracy: 0.9704 - f1: 0.9720 - loss: 0.0757 - val_accuracy: 0.9700 - val_f1:
Epoch 19/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 197s 2s/step - accuracy: 0.9701 - f1: 0.9717 - loss: 0.0774 - val_accuracy: 0.9721 - val_f1:
Epoch 20/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 200s 2s/step - accuracy: 0.9707 - f1: 0.9723 - loss: 0.0754 - val_accuracy: 0.9731 - val_f1:
Epoch 21/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 195s 2s/step - accuracy: 0.9681 - f1: 0.9697 - loss: 0.0783 - val_accuracy: 0.9707 - val_f1:
Epoch 22/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 197s 2s/step - accuracy: 0.9682 - f1: 0.9699 - loss: 0.0825 - val_accuracy: 0.9721 - val_f1:
Epoch 23/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 191s 2s/step - accuracy: 0.9710 - f1: 0.9726 - loss: 0.0758 - val_accuracy: 0.9718 - val_f1:
Epoch 24/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 194s 2s/step - accuracy: 0.9683 - f1: 0.9700 - loss: 0.0800 - val_accuracy: 0.9695 - val_f1:
Epoch 25/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 189s 2s/step - accuracy: 0.9727 - f1: 0.9742 - loss: 0.0685 - val_accuracy: 0.9718 - val_f1:
Epoch 26/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 194s 2s/step - accuracy: 0.9739 - f1: 0.9754 - loss: 0.0659 - val_accuracy: 0.9723 - val_f1:
Epoch 27/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 196s 2s/step - accuracy: 0.9724 - f1: 0.9739 - loss: 0.0695 - val_accuracy: 0.9713 - val_f1:
Epoch 28/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 191s 2s/step - accuracy: 0.9729 - f1: 0.9743 - loss: 0.0695 - val_accuracy: 0.9719 - val_f1:
Epoch 29/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 192s 2s/step - accuracy: 0.9724 - f1: 0.9739 - loss: 0.0697 - val_accuracy: 0.9730 - val_f1:
Epoch 30/40
125/125 ━━━━━━━━━━━━━━━━━━━━ 196s 2s/step - accuracy: 0.9733 - f1: 0.9747 - loss: 0.0678 - val_accuracy: 0.9716 - val_f1:
```

```
# Cell 15 — plot training history
hist = pd.DataFrame(history.history)
fig, ax = plt.subplots(1,3, figsize=(17,4))
for metric, title in [('loss','Loss'),('accuracy','Pixel accuracy'),('f1','F1')]:
    ax[['loss','accuracy','f1'].index(metric)].plot(hist[metric], label='train')
    if 'val_'+metric in hist: ax[['loss','accuracy','f1'].index(metric)].plot(hist['val_'+metric], label='validation')
    ax[['loss','accuracy','f1'].index(metric)].set_title(title); ax[['loss','accuracy','f1'].index(metric)].legend(); ax[[
plt.tight_layout(); plt.savefig(RUN_DIR/'training_curves.png', dpi=160)
```



Double-click (or enter) to edit

Threshold selection and held-out test evaluation

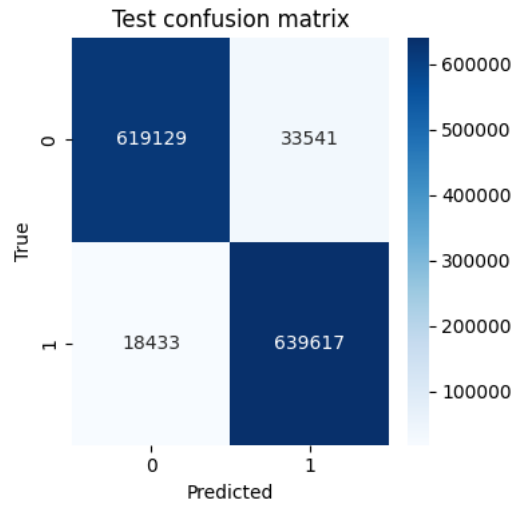
The model outputs logits. A threshold is selected by maximizing validation F1 over a grid. This threshold is then frozen before the test set is touched. The test metrics below are therefore not used for checkpoint or threshold selection.

```
# Cell 16 — collect predictions and tune threshold on validation only

def collect_predictions(seq):
    ys, ps = [], []
    for i in tqdm(range(len(seq)), desc=seq.split):
```

```
        x,y = seq[i]; p = tf.nn.sigmoid(model(x, training=False)).numpy()
        ys.append(y.reshape(-1)); ps.append(p.reshape(-1))
    return np.concatenate(ys).astype(np.uint8), np.concatenate(ps)

y_val, p_val = collect_predictions(val_seq)
thresholds = np.linspace(0.10,0.90,81)
f1s = [precision_recall_fscore_support(y_val, p_val>=t, average='binary', zero_division=0)[2] for t in thresholds]
best_threshold = float(thresholds[int(np.argmax(f1s))])
print('Best validation threshold:', best_threshold, 'F1:', max(f1s))
y_test, p_test = collect_predictions(test_seq)
pred_test = p_test >= best_threshold
precision, recall, f1, _ = precision_recall_fscore_support(y_test, pred_test, average='binary', zero_division=0)
metrics = {'threshold_selected_on':'validation', 'threshold':best_threshold, 'accuracy':float(accuracy_score(y_test,pred_te
print(json.dumps(metrics, indent=2))
(RUN_DIR/'test_metrics.json').write_text(json.dumps(metrics, indent=2))
```

```
val: 100%  25/25 [00:53<00:00, 2.33s/it]
WARNING:tensorflow:5 out of the last 5 calls to <function conv.<locals>._conv_xla at 0x7922beff4180> triggered tf.function r
WARNING:tensorflow:6 out of the last 6 calls to <function conv.<locals>._conv_xla at 0x7922beff4180> triggered tf.function r
Best validation threshold: 0.53 F1: 0.9743620567134291
test: 100%  5/5 [00:09<00:00, 1.83s/it]
{
  "threshold_selected_on": "validation",
  "threshold": 0.53,
  "accuracy": 0.9603469848632813,
  "precision": 0.9501736590815232,
  "recall": 0.9719884507256288,
  "f1": 0.9609572658818156,
  "iou": 0.9248486460928497
}
222
```

```
# Cell 17 — confusion matrix and qualitative test predictions
cm = confusion_matrix(y_test, pred_test)
plt.figure(figsize=(4,4)); sns.heatmap(cm, annot=True, fmt='d', cmap='Blues'); plt.xlabel('Predicted'); plt.ylabel('True');

with h5py.File(CACHE_PATH,'r') as h: test_x = h['test/images'][:]
probs = p_test.reshape(len(test_x), *TARGET_SIZE)
for i in range(min(8,len(test_x))):
    fig, ax = plt.subplots(1,4, figsize=(14,3.5))
    ax[0].imshow(test_x[i][...,:3]); ax[0].set_title('RGB')
    ax[1].imshow(test_x[i][...,3], cmap='magma'); ax[1].set_title('NIR')
    ax[2].imshow(probs[i], cmap='viridis', vmin=0,vmax=1); ax[2].set_title('Probability')
    ax[3].imshow(probs[i]>=best_threshold, cmap='gray'); ax[3].set_title('Binary mask')
    for a in ax: a.axis('off')
    plt.tight_layout(); plt.savefig(PRED_DIR/f'{DATASET}_test_{i:03d}.png', dpi=160); plt.close(fig)
```



Reproducibility notes and recommended experiments

The principal comparison should be made against the earlier attention U-Net and TransUNet++ results using the same official split and the same metric definitions. Report the selected threshold, image resolution, augmentation policy, number of trainable parameters, and whether the result is from validation or test data.

For a stronger study, repeat the run with three seeds and report mean ± standard deviation. Add an ablation table for the spectral stem, transformer bottleneck, attention gates, and augmentation. If the raw archive's folder names differ from the expected layout, only the

`infer_split` and `discover_pairs` cells should need adjustment. Never select a checkpoint or threshold using test labels.

The 4-band Amazon archive is large. For an initial smoke test, set `TARGET_SIZE=(128,128)`, `EPOCHS=2`, and `BATCH_SIZE=2`; then delete or change `CACHE_PATH` before the full experiment. Keep the full-resolution results separate from smoke-test results.

References

```
# Save the complete trained TensorFlow/Keras model

FULL_MODEL_PATH = RUN_DIR / "trained_model.keras"

model.save(FULL_MODEL_PATH)

print("Complete model saved to:")
print(FULL_MODEL_PATH)
```

```
Complete model saved to:
/content/drive/MyDrive/deforestation_segmentation_zenodo/runs/20260904_104701/trained_model.keras
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

```
# Load the complete model in a future Colab session
#FULL_MODEL_PATH = RUN_DIR / "trained_model.keras"
def bce_dice(y_true, logits):
    bce = tf.reduce_mean(tf.nn.sigmoid_cross_entropy_with_logits(labels=y_true, logits=logits))
    return 0.5*bce + 0.5*dice_loss(y_true, logits)

class SegmentationMetrics(keras.metrics.Metric):
    def __init__(self, name='seg_metrics', threshold=0.5, **kwargs):
        super().__init__(name=name, **kwargs); self.threshold=threshold
        self.tp=self.add_weight(name='tp', shape=(), initializer='zeros'); self.fp=self.add_weight(name='fp', shape=(), init
    def update_state(self,y_true,y_pred,sample_weight=None):
        p=tf.cast(tf.nn.sigmoid(y_pred)>=self.threshold,tf.float32); y=tf.cast(y_true>=0.5,tf.float32)
        self.tp.assign_add(tf.reduce_sum(p*y)); self.fp.assign_add(tf.reduce_sum(p*(1-y))); self.fn.assign_add(tf.reduce_sum
    def result(self):
        precision=tf.math.divide_no_nan(self.tp,self.tp+self.fp); recall=tf.math.divide_no_nan(self.tp,self.tp+self.fn)
        return tf.math.divide_no_nan(2*precision*recall,precision+recall)
    def reset_state(self):
        for v in [self.tp,self.fp,self.fn]: v.assign(0.)

loaded_model = keras.models.load_model(
    FULL_MODEL_PATH,
    custom_objects={
        "bce_dice": bce_dice,
        "SegmentationMetrics": SegmentationMetrics
    }
)

print("Complete model loaded successfully.")
```

```
Complete model loaded successfully.
```

```
# Visualize test-set predictions

import numpy as np
import matplotlib.pyplot as plt
import h5py
from pathlib import Path

# Load test images and ground-truth masks from the HDF5 cache
with h5py.File(CACHE_PATH, "r") as h:
    test_images = h["test/images"][:]
    test_masks = h["test/masks"][:]
```

```
print("Test images:", test_images.shape)
print("Test masks:", test_masks.shape)

# p_test was generated in the evaluation cell.
# It contains flattened sigmoid probabilities.
# Convert it back to image dimensions.
test_probabilities = p_test.reshape(
    len(test_images),
    TARGET_SIZE[0],
    TARGET_SIZE[1]
)

# Use the validation-selected threshold.
# If best_threshold does not exist, use 0.53 from your result.
try:
    display_threshold = best_threshold
except NameError:
    display_threshold = 0.53

test_predictions = (
    test_probabilities >= display_threshold
).astype(np.uint8)

# Number of test images to display
num_to_display = min(20, len(test_images))

for i in range(num_to_display):

    image = test_images[i]

    # RGB visualization.
    # The first three channels are treated as B4, B3, B2.
    rgb = image[..., :3]

    # NIR channel is the fourth channel.
    nir = image[..., 3]

    ground_truth = test_masks[i, ..., 0]
    probability = test_probabilities[i]
    prediction = test_predictions[i]

    # Error map:
    # True positive  = green
    # False positive = red
    # False negative = blue
    true_positive = (prediction == 1) & (ground_truth == 1)
    false_positive = (prediction == 1) & (ground_truth == 0)
    false_negative = (prediction == 0) & (ground_truth == 1)

    error_map = np.zeros((*ground_truth.shape, 3), dtype=np.float32)
    error_map[true_positive] = [0.0, 1.0, 0.0]
    error_map[false_positive] = [1.0, 0.0, 0.0]
    error_map[false_negative] = [0.0, 0.3, 1.0]

    # RGB overlay of the predicted mask.
    overlay = rgb.copy()
    overlay[prediction == 1] = (
        0.55 * overlay[prediction == 1]
        + 0.45 * np.array([1.0, 0.0, 0.0])
    )

    fig, axes = plt.subplots(2, 4, figsize=(18, 9))

    axes[0, 0].imshow(rgb)
    axes[0, 0].set_title(f"Test image {i} — RGB")

    axes[0, 1].imshow(nir, cmap="magma")
    axes[0, 1].set_title("NIR band")

    axes[0, 2].imshow(ground_truth, cmap="gray", vmin=0, vmax=1)
    axes[0, 2].set_title("Ground-truth mask")

    axes[0, 3].imshow(probability, cmap="viridis", vmin=0, vmax=1)
    axes[0, 3].set_title("Predicted probability")

    axes[1, 0].imshow(prediction, cmap="gray", vmin=0, vmax=1)
    axes[1, 0].set_title(
        f"Predicted mask\nthreshold = {display_threshold:.2f}"
    )

    axes[1, 1].imshow(overlay)
    axes[1, 1].set_title("Prediction overlay")
```

```
axes[1, 2].imshow(error_map)
axes[1, 2].set_title(
    "Error map\n"
    "Green=TP, Red=FP, Blue=FN"
)

axes[1, 3].axis("off")

for ax in axes.flat:
    ax.axis("off")

plt.tight_layout()
plt.show()
```